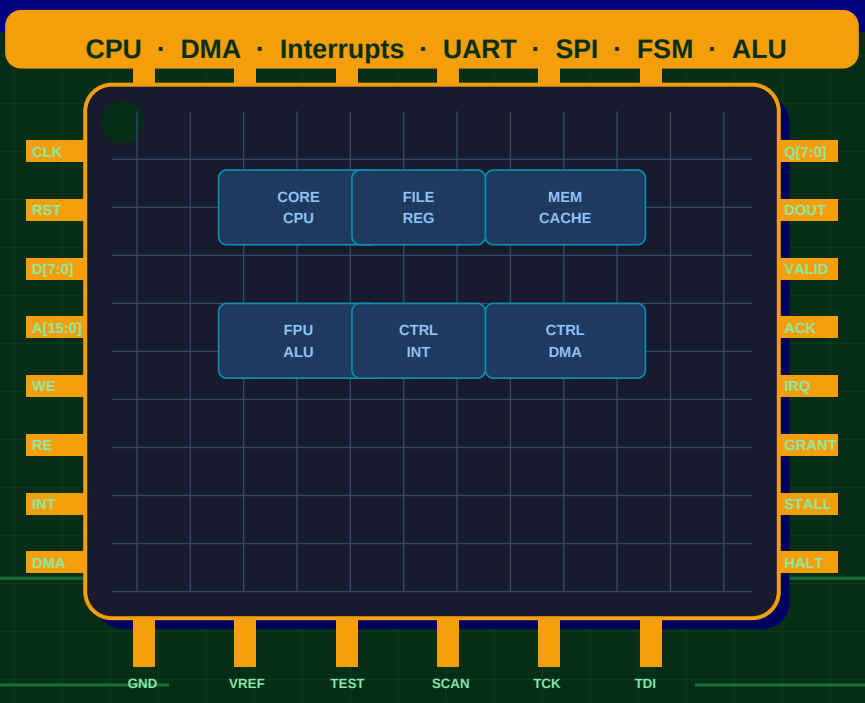
Verilog HDL

Complete Notes · Basics to Advanced RTL Design



Designed By Naravamakula Karthik Reddy

Verilog HDL | Digital Design | RTL Engineering

Complete Reference — Basic to Advanced • 20 Chapters • Real-World Projects

■ Table of Contents

1 Introduction to Verilog

Verilog is a Hardware Description Language (HDL) used to model, simulate, and synthesize digital circuits. Created in 1984 at Gateway Design Automation, standardized as IEEE 1364-1995, updated in Verilog-2001, and extended to SystemVerilog (IEEE 1800). It is used in every chip — CPUs, GPUs, FPGAs, ASICs.

Software vs Hardware

Concept	Software (C/Python)	Verilog (HDL)
Execution	Sequential — one line at a time	Parallel — all hardware runs at once
Runs on	CPU / Processor	FPGA / ASIC Silicon
Timing	OS-managed time	Nanosecond precision
Variables	RAM memory locations	Physical wires and registers
Output	Executable binary	Synthesized gate-level netlist
Debugging	Print statements / IDE debugger	Simulation waveforms (GTKWave)

Design Flow

1. Spec → 2. RTL Coding (Verilog) → 3. Simulation (Icarus/ModelSim) → 4. Synthesis (Vivado/Quartus) → 5. Place & Route → 6. Timing Analysis → 7. FPGA Programming

Abstraction Levels

Level	What it models	Example
Behavioral	High-level functional description	always + if/case (most common RTL)
RTL	Register transfer operations	Clocked always blocks
Gate	Logic gate connections	and, or, not primitives
Switch	Transistor switches	nmos, pmos (rarely used)

2 Basic Syntax & Structure

Module — The Building Block

Verilog

```verilog
`timescale 1ns / 1ps          // time unit / precision

// Single-line comment
/* Multi-line comment */

module my_module #(
    parameter WIDTH = 8        // parameter with default
) (
    input              clk,    // 1-bit input
    input              rst_n,  // active-low reset
    input  [WIDTH-1:0] data_in,
    output reg [WIDTH-1:0] data_out,
    output wire        valid
);
    wire [WIDTH-1:0] temp;
    reg  [3:0]       counter;

    // logic here ...

endmodule
```

Naming Conventions

Convention	Usage	Example
snake_case	Signal names	data_valid, byte_count
UPPER_CASE	Parameters	DATA_WIDTH, MAX_ADDR
_n suffix	Active-low signals	rst_n, cs_n, oe_n
_i / _o suffix	Input / Output ports	clk_i, data_o
u_ prefix	Module instances	u_alu, u_fifo
s_ prefix	State registers	s_state, s_next

3 Data Types

Verilog

```
// ■■ Net types ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
wire        y;              // 1-bit connection (most common)
wire [7:0]  data_bus;       // 8-bit bus
tri  [7:0]  tristate_bus;   // tri-state bus
supply1     vdd;            // tied to logic 1
supply0     gnd;            // tied to logic 0

// ■■ Variable types ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
reg         q;              // 1-bit storage
reg  [7:0]  byte_val;       // 8-bit register
reg  [31:0] word;           // 32-bit
reg  signed [15:0] signed_val; // signed 16-bit

integer     i, j;           // 32-bit signed (for loops)
real        pi = 3.14159;   // 64-bit float (sim only)
time        t_start;        // 64-bit sim time

// ■■ Arrays (Memory) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
reg [7:0]   mem   [0:255];  // 256 x 8-bit RAM
reg [15:0]  rom   [0:1023]; // 1024 x 16-bit ROM
reg [3:0]   lut   [0:15];   // 16-entry lookup table
```

Four-Value Logic

Value	Meaning	Typical Source
0	Logic zero	GND, driven low, reset
1	Logic one	VDD, driven high
x	Unknown	Uninitialized reg, bus conflict
z	High impedance	Undriven wire, tri-state output

4 Operators

Verilog

```
// ■■ Arithmetic ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
assign sum   = a + b;        assign diff = a - b;
assign prod  = a * b;        assign quot = a / b;
assign rem   = a % b;

// ■■ Bitwise (bit-by-bit) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
assign y_and  = a & b;       assign y_or  = a | b;
assign y_xor  = a ^ b;       assign y_not = ~a;
assign y_xnor = a ~^ b;

// ■■ Logical (returns 1-bit true/false) ■■■■■■■■■■■■■■■■■■■■■■■■
assign l_and = a && b;       assign l_or  = a || b;
assign l_not = !a;

// ■■ Relational ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
assign eq = (a == b);        assign neq = (a != b);
assign lt = (a <  b);        assign gt  = (a >  b);

// ■■ Shift ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
assign sl = a << 2;          // logical left  (fill 0)
assign sr = a >> 1;          // logical right (fill 0)
assign sa = $signed(a) >>> 1;// arithmetic right (fill sign)

// ■■ Reduction (1-bit result) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
assign parity   = ^data;     // XOR all bits → odd-parity check
assign all_ones = &data;     // AND all bits → 1 if all 1s
assign any_one  = |data;     // OR  all bits → 1 if any 1

// ■■ Ternary ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
assign y = sel ? a : b;      // 2:1 MUX

// ■■ Concatenation & Replication ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
assign byte_val = {hi, lo};  // join signals
assign bus16    = {2{byte_val}};   // replicate 2x
```

5 Numbers & Constants

Verilog

```verilog
// Format: <width>'<base><value>
4'b1010           // 4-bit binary      = decimal 10
8'hFF             // 8-bit hex         = 255
8'd255            // 8-bit decimal     = 255
32'h0000_FFFF     // underscore for readability
1'bx              // unknown
1'bz              // high-impedance

// ■■ parameter ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
parameter DATA_W  = 8;
parameter MAX_CNT = 255;
parameter RESET   = 8'h00;

// ■■ localparam (cannot be overridden externally) ■■■■■■■■■■
localparam IDLE  = 2'b00;
localparam READ  = 2'b01;
localparam WRITE = 2'b10;
localparam ERROR = 2'b11;

// ■■ `define (file-level text macro) ■■■■■■■■■■■■■■■■■■■■■■■■
`define CLK_PERIOD 10
`define TRUE  1'b1
`define FALSE 1'b0
```

6 Gate-Level Modeling

Verilog

```verilog
module gates_example (input a, b, c, output y1,y2,y3,y4,y5);
    and   g1 (y1, a, b);           // AND
    or    g2 (y2, a, b);           // OR
    not   g3 (y3, a);              // NOT
    nand  g4 (y4, a, b);           // NAND
    xor   g5 (y5, a, b, c);        // 3-input XOR
    and   #(5) g6 (y6, a, b);      // 5 ns delay
    and   #(3,5) g7 (y7, a, b);    // rise=3ns fall=5ns
endmodule
```

Gate	Output is HIGH when…	Use Case
and	ALL inputs HIGH	Enable / mask logic
or	ANY input HIGH	Priority / OR bus
not	Input is LOW	Signal inversion
nand	NOT all inputs HIGH	Universal gate (saves area)
xor	Inputs DIFFERENT	Parity, adder sum
xnor	Inputs SAME	Equality comparator
buf	Always = input	Drive strength booster

7 Dataflow Modeling

Verilog

```
// Continuous assign — always active
assign y       = a & b;
assign sum     = a ^ b ^ cin;
assign carry   = (a&b) | (b&cin) | (a&cin);
assign out     = sel ? a : b;          // 2:1 MUX
assign {co,s}  = a + b + cin;          // full adder
assign sign_ext= {{8{data[7]}}, data}; // sign-extend 8→16
assign parity  = ^data_byte;           // parity bit

// 4-bit ripple carry adder
module full_adder (input a,b,cin, output sum,cout);
    assign sum  = a ^ b ^ cin;
    assign cout = (a&b)|(b&cin)|(a&cin);
endmodule

module rca4 (input [3:0] a,b, input cin, output [3:0] sum, output cout);
    wire c1,c2,c3;
    full_adder fa0(.a(a[0]),.b(b[0]),.cin(cin), .sum(sum[0]),.cout(c1));
    full_adder fa1(.a(a[1]),.b(b[1]),.cin(c1),  .sum(sum[1]),.cout(c2));
    full_adder fa2(.a(a[2]),.b(b[2]),.cin(c2),  .sum(sum[2]),.cout(c3));
    full_adder fa3(.a(a[3]),.b(b[3]),.cin(c3),  .sum(sum[3]),.cout(cout));
endmodule
```

8 Behavioral Modeling

always Blocks

Verilog

```verilog
// ■■ Combinational — use @(*) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
always @(*) begin
    y = a & b;              // blocking = for combinational
end

// ■■ Sequential — posedge/negedge ■■■■■■■■■■■■■■■■■■■■■■■■■■
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) q <= 1'b0;  // async active-low reset
    else        q <= d;     // capture on rising edge
end
```

Blocking vs Non-Blocking

Verilog

```verilog
// Blocking =  (combinational always)
always @(*) begin
    temp = a + b;           // executes immediately
    out  = temp & mask;     // uses updated temp
end

// Non-Blocking <=  (sequential always)
// ALL right-hand sides evaluated FIRST, then assigned
always @(posedge clk) begin
    q1 <= d;                // shift register: q1 gets d
    q2 <= q1;               // q2 gets OLD q1 (correct!)
end
```

■ **Warning: Never mix = and <= in the same always block. Use = for combinational, <= for clocked. Violation #1 bug source.**

9 Conditional Statements

Verilog

```
// ■■ if-else ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
always @(*) begin
    if      (a > b) begin out = a; valid = 1'b1; end
    else if (a == b) begin out = 0; valid = 1'b0; end
    else             begin out = b; valid = 1'b1; end
end

// ■■ case (exact match) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
always @(*) begin
    case (sel)
        2'b00: y = a;
        2'b01: y = b;
        2'b10: y = c;
        2'b11: y = d;
        default: y = 4'bx; // ALWAYS include default
    endcase
end

// ■■ casex (x/? = don't care in pattern) ■■■■■■■■■■■■■■■■■
always @(*) begin
    casex (opcode)
        4'b1???: result = op_a;   // matches 1xxx
        4'b01??: result = op_b;
        default: result = 8'h00;
    endcase
end

// ■■ casez (z/? = don't care — safer) ■■■■■■■■■■■■■■■■■■■■■
always @(*) begin
    casez (priority_in)
        4'b1???: out = 2'd3;      // highest priority
        4'b01??: out = 2'd2;
        4'b001?: out = 2'd1;
        4'b0001: out = 2'd0;
        default: out = 2'bx;
    endcase
end
```

10 Loops

Verilog

```verilog
// ■■ for loop (synthesizable, unrolled to hardware) ■■■■■■■■
integer i;
always @(*) begin
    for (i = 0; i < 8; i = i + 1)
        out[i] = in_a[i] & in_b[i];   // 8 AND gates in parallel
end

// ■■ while (bounded — synthesizable) ■■■■■■■■■■■■■■■■■■■■■■
integer k = 0;
always @(*) begin
    result = 0;
    k = 0;
    while (k < DATA_W) begin
        result = result + data[k];
        k = k + 1;
    end
end

// ■■ repeat (fixed iterations) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
initial begin
    repeat(8) @(posedge clk);   // wait 8 clock cycles
end

// ■■ forever (testbench clock) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
initial begin
    clk = 0;
    forever #5 clk = ~clk;      // 100 MHz clock
end
```

■ **Tip: Use for loops with constant bounds for synthesizable code. The synthesis tool unrolls them into parallel hardware. forever/while with #delays are simulation-only.**

11 Sequential Circuits

Flip-Flop Variants

Verilog

```verilog
// D FF — async active-low reset
module dff (input clk, rst_n, d, output reg q);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) q <= 1'b0;
        else        q <= d;
    end
endmodule

// D FF — sync reset + enable
module dff_en (input clk, rst_n, en, d, output reg q);
    always @(posedge clk) begin
        if (!rst_n)  q <= 1'b0;
        else if (en) q <= d;
    end
endmodule

// N-bit register
module reg_N #(parameter N=8) (
    input clk,rst_n,load, input [N-1:0] din, output reg [N-1:0] dout);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)    dout <= {N{1'b0}};
        else if (load) dout <= din;
    end
endmodule
```

Counter & Shift Register

Verilog

```verilog
// Up/Down counter
module counter #(parameter N=4) (
    input clk,rst_n,en,up_down, output reg [N-1:0] count);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)     count <= {N{1'b0}};
        else if (en)    count <= up_down ? count+1 : count-1;
    end
endmodule

// 8-bit SIPO shift register
module sipo (input clk,rst_n,si, output reg [7:0] po);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) po <= 8'h00;
        else        po <= {po[6:0], si}; // shift left, feed serial in
    end
endmodule
```

12 Finite State Machines (FSM)

Type	Output depends on	Timing
Moore	Current state only	Slightly slower, more stable outputs
Mealy	State AND inputs	Faster response, can glitch

3-Block Moore FSM — Traffic Light

Verilog

```verilog
module traffic_light (input clk, rst_n, output reg [1:0] light);
    localparam [1:0] S_RED=2'b00, S_GREEN=2'b01, S_YELLOW=2'b10;
    reg [1:0] state, next_state;

    // BLOCK 1: State register (sequential)
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) state <= S_RED;
        else        state <= next_state;
    end

    // BLOCK 2: Next-state logic (combinational)
    always @(*) begin
        case (state)
            S_RED:    next_state = S_GREEN;
            S_GREEN:  next_state = S_YELLOW;
            S_YELLOW: next_state = S_RED;
            default:  next_state = S_RED;
        endcase
    end

    // BLOCK 3: Output logic — Moore (state only)
    always @(*) begin
        case (state)
            S_RED:    light = 2'b00;
            S_GREEN:  light = 2'b10;
            S_YELLOW: light = 2'b01;
            default:  light = 2'b00;
        endcase
    end
endmodule
```

13 Hierarchical Design

Verilog

```
module top (input clk,rst_n, input [7:0] a,b, output [7:0] result);
    wire [7:0] sum_w, alu_out;

    // Named port mapping — ALWAYS use this
    adder_8bit u_adder (
        .clk(clk), .a(a), .b(b),
        .sum(sum_w), .cout()      // unconnected — leave empty
    );

    // With parameter override
    alu #(.WIDTH(8)) u_alu (
        .clk(clk), .rst_n(rst_n),
        .op_a(a), .op_b(b), .result(alu_out)
    );

    assign result = alu_out;
endmodule
```

14 Parameters & Generate Blocks

Verilog

```verilog
// Parameterized module
module counter #(parameter N=4, INIT=0) (
    input clk,rst_n,en, output reg [N-1:0] count);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)  count <= INIT[N-1:0];
        else if (en) count <= count + 1;
    end
endmodule

// Instantiate with different widths
counter #(.N(4))  c4  (.clk(clk),.rst_n(rst_n),.en(1'b1),.count(c4o));
counter #(.N(8))  c8  (.clk(clk),.rst_n(rst_n),.en(1'b1),.count(c8o));
counter #(.N(16)) c16 (.clk(clk),.rst_n(rst_n),.en(en16),.count(c16o));

// Generate: 8 flip-flop chain
module ff_chain #(parameter N=8) (input clk,rst_n,d, output q);
    wire [N-1:0] chain;
    genvar i;
    generate
        for (i=0; i<N; i=i+1) begin : ff_gen
            if (i==0) dff u (.clk(clk),.rst_n(rst_n),.d(d),       .q(chain[0]));
            else      dff u (.clk(clk),.rst_n(rst_n),.d(chain[i-1]),.q(chain[i]));
        end
    endgenerate
    assign q = chain[N-1];
endmodule
```

15 Tasks & Functions

Verilog

```verilog
// Function — returns a value, NO delays allowed
function [7:0] add_bytes;
    input [7:0] a, b;
    begin add_bytes = a + b; end
endfunction

// Automatic recursive function
function automatic [31:0] factorial;
    input [4:0] n;
    begin
        if (n <= 1) factorial = 1;
        else        factorial = n * factorial(n-1);
    end
endfunction

// Task — multiple outputs, delays allowed
task apply_reset;
    input [7:0] hold_cycles;
    begin
        rst_n = 1'b0;
        repeat(hold_cycles) @(posedge clk);
        @(posedge clk); rst_n = 1'b1;
        $display("[%0t] Reset released", $time);
    end
endtask

// Usage
result = add_bytes(8'hAA, 8'h11);
initial begin apply_reset(8'd5); end
```

16 Testbenches & Verification

Verilog

```verilog
`timescale 1ns/1ps
module tb_counter;
    reg  clk, rst_n, en, up_down;
    wire [3:0] count;
    integer fail_count = 0;

    counter #(.N(4)) dut (
        .clk(clk), .rst_n(rst_n), .en(en),
        .up_down(up_down), .count(count)
    );

    initial  clk = 1'b0;
    always  #5 clk = ~clk;    // 100 MHz

    initial begin
        $dumpfile("tb.vcd"); $dumpvars(0, tb_counter);
        {rst_n,en,up_down} = 0; #22;
        rst_n=1; en=1; up_down=1;        // count up
        repeat(20) @(posedge clk);
        up_down=0;                        // count down
        repeat(10) @(posedge clk);
        if (fail_count==0) $display("[PASS] All tests passed!");
        $finish;
    end
    initial $monitor("t=%0t rst=%b en=%b count=%0d",$time,rst_n,en,count);
endmodule
```

System Tasks Reference

Task	Description	Example
$display	Print once now	$display("val=%d", x)
$monitor	Print on change	$monitor("%t %d", $time, q)
$finish	End simulation	$finish;
$stop	Pause sim	$stop;
$dumpfile	VCD filename	$dumpfile("out.vcd")
$dumpvars	Dump signals	$dumpvars(0, tb)
$readmemh	Load hex to mem	$readmemh("file.hex",mem)
$random	Random number	data = $random % 256

17 Compiler Directives

Verilog

```
`timescale 1ns / 1ps           // unit / precision
`default_nettype none          // catch undeclared wires (BEST PRACTICE)

`define CLK_PERIOD 10
`define TRUE  1'b1
`define FALSE 1'b0

`include "definitions.vh"

`ifdef SIMULATION
    initial $display("SIMULATION mode");
`elsif FPGA_TARGET
    // FPGA-specific instantiation
`else
    // ASIC path
`endif

`ifndef MY_DEFINES_VH
`define MY_DEFINES_VH
    // guard to prevent double-include
`endif

always #(`CLK_PERIOD/2) clk = ~clk;   // uses define
```

18 Memory Modeling

Synchronous RAM

Verilog

```
module spram #(parameter AW=8, DW=8) (
    input             clk, we,
    input  [AW-1:0]   addr,
    input  [DW-1:0]   wdata,
    output reg [DW-1:0] rdata
);
    reg [DW-1:0] mem [0:(2**AW)-1];
    always @(posedge clk) begin
        if (we) mem[addr] <= wdata;
        rdata <= mem[addr];
    end
endmodule
```

ROM with File Load

Verilog

```
module rom_256x8 (input [7:0] addr, output reg [7:0] data);
    reg [7:0] rom_mem [0:255];
    initial $readmemh("rom.hex", rom_mem);
    always @(*) data = rom_mem[addr];
endmodule
```

Synchronous FIFO

Verilog

```verilog
module fifo #(parameter DW=8, DEPTH=16) (
    input          clk, rst_n,
    input          wr_en, rd_en,
    input  [DW-1:0] din,
    output reg [DW-1:0] dout,
    output         full, empty
);
    localparam PTR_W = $clog2(DEPTH);
    reg [DW-1:0]    mem   [0:DEPTH-1];
    reg [PTR_W:0]   wr_ptr, rd_ptr, count;

    assign full  = (count == DEPTH);
    assign empty = (count == 0);

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            wr_ptr<=0; rd_ptr<=0; count<=0;
        end else begin
            if (wr_en && !full)  begin mem[wr_ptr[PTR_W-1:0]] <= din;  wr_ptr<=wr_ptr+1; count<=
            if (rd_en && !empty) begin dout <= mem[rd_ptr[PTR_W-1:0]]; rd_ptr<=rd_ptr+1; count<=
        end
    end
endmodule
```

CDC 2-FF Synchronizer

Verilog

```verilog
module cdc_sync #(parameter STAGES=2) (
    input clk_dst, rst_n, async_in,
    output sync_out
);
    reg [STAGES-1:0] sync_ff;
    always @(posedge clk_dst or negedge rst_n) begin
        if (!rst_n) sync_ff <= {STAGES{1'b0}};
        else        sync_ff <= {sync_ff[STAGES-2:0], async_in};
    end
    assign sync_out = sync_ff[STAGES-1];
endmodule
```

19 Real-World RTL Projects

19.1 UART Transmitter

Verilog

```verilog
module uart_tx #(parameter CLK_FREQ=50_000_000, BAUD=115200) (
    input        clk, rst_n,
    input        tx_valid,
    input  [7:0] tx_data,
    output reg   tx,
    output reg   tx_ready
);
    localparam CLKS_PER_BIT = CLK_FREQ / BAUD;  // ~434 for 50MHz/115200
    localparam [2:0] IDLE=0,START=1,DATA=2,STOP=3;

    reg [2:0]  state;
    reg [15:0] clk_cnt;
    reg [2:0]  bit_idx;
    reg [7:0]  tx_shift;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state<=IDLE; tx<=1; tx_ready<=1; clk_cnt<=0; bit_idx<=0;
        end else begin
            case (state)
                IDLE: begin
                    tx<=1; tx_ready<=1; clk_cnt<=0;
                    if (tx_valid) begin
                        tx_shift<=tx_data; tx_ready<=0; state<=START;
                    end
                end
                START: begin
                    tx<=0;   // start bit
                    if (clk_cnt==CLKS_PER_BIT-1) begin clk_cnt<=0; state<=DATA; end
                    else clk_cnt<=clk_cnt+1;
                end
                DATA: begin
                    tx<=tx_shift[bit_idx];
                    if (clk_cnt==CLKS_PER_BIT-1) begin
                        clk_cnt<=0;
                        if (bit_idx==7) begin bit_idx<=0; state<=STOP; end
                        else bit_idx<=bit_idx+1;
                    end else clk_cnt<=clk_cnt+1;
                end
                STOP: begin
                    tx<=1;   // stop bit
                    if (clk_cnt==CLKS_PER_BIT-1) begin
                        clk_cnt<=0; tx_ready<=1; state<=IDLE;
                    end else clk_cnt<=clk_cnt+1;
                end
```

19.2 SPI Master (CPOL=0 CPHA=0)

Verilog

```verilog
module spi_master #(parameter CLK_DIV=4) (
    input        clk, rst_n, start,
    input  [7:0] mosi_data,
    output reg   mosi, sclk, cs_n,
    input        miso,
    output reg [7:0] miso_data,
    output reg   done
);
    reg [3:0]  bit_cnt;
    reg [7:0]  shift_reg;
    reg [2:0]  clk_div;
    reg        active;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            sclk<=0; cs_n<=1; mosi<=0; done<=0; active<=0; bit_cnt<=0;
        end else begin
            done<=0;
            if (!active && start) begin
                active<=1; cs_n<=0; shift_reg<=mosi_data; bit_cnt<=0; clk_div<=0;
            end else if (active) begin
                clk_div<=clk_div+1;
                if (clk_div==CLK_DIV/2-1) begin
                    sclk<=~sclk;
                    if (!sclk) begin  // rising edge – sample MISO
                        miso_data<={miso_data[6:0],miso};
                    end else begin    // falling edge – shift MOSI
                        mosi<=shift_reg[7]; shift_reg<={shift_reg[6:0],1'b0};
                        if (bit_cnt==7) begin
                            active<=0; cs_n<=1; done<=1;
                        end else bit_cnt<=bit_cnt+1;
                    end
                    clk_div<=0;
                end
            end
        end
    end
endmodule
```

19.3 Interrupt Controller (8-source priority)

Verilog

```verilog
module interrupt_ctrl #(parameter N=8) (
    input              clk, rst_n,
    input  [N-1:0]     irq_lines,    // interrupt request lines
    input  [N-1:0]     irq_mask,     // 1 = masked (disabled)
    input              irq_ack,      // CPU acknowledges interrupt
    output reg [N-1:0] irq_pending,  // pending interrupt register
    output reg [$clog2(N)-1:0] irq_id, // ID of highest-priority active IRQ
    output reg         irq_out       // interrupt request to CPU
);
    wire [N-1:0] irq_active = irq_pending & ~irq_mask;
    integer i;

    // Latch rising edges of IRQ lines
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            irq_pending <= {N{1'b0}};
        end else begin
            // Set on rising edge of irq_lines
            irq_pending <= irq_pending | irq_lines;
            // Clear acknowledged IRQ
            if (irq_ack) irq_pending[irq_id] <= 1'b0;
        end
    end

    // Priority encoder — IRQ 0 = highest priority
    always @(*) begin
        irq_out = 1'b0;
        irq_id  = {$clog2(N){1'b0}};
        for (i = N-1; i >= 0; i = i-1) begin
            if (irq_active[i]) begin
                irq_out = 1'b1;
                irq_id  = i[$clog2(N)-1:0];
            end
        end
    end
endmodule
```

19.4 DMA Controller (Single-Channel)

Verilog

```verilog
module dma_ctrl (
    input         clk, rst_n,
    // CPU programming interface
    input         cfg_wr,
    input  [1:0]  cfg_addr,
    input  [31:0] cfg_data,
    // DMA trigger
    input         dma_req,
    output reg    dma_ack,
    // Bus master interface
    output reg        bus_req,
    input             bus_gnt,
    output reg [31:0] bus_addr,
    output reg [31:0] bus_wdata,
    output reg        bus_we,
    input      [31:0] bus_rdata,
    input             bus_ready,
    output reg        dma_done
);
    localparam IDLE=2'd0, WAIT_GNT=2'd1, TRANSFER=2'd2, DONE=2'd3;
    reg [1:0]  state;
    reg [31:0] src_addr, dst_addr;
    reg [15:0] length, cnt;

    // CPU writes config registers
    always @(posedge clk) begin
        if (cfg_wr) begin
            case (cfg_addr)
                2'd0: src_addr <= cfg_data;
                2'd1: dst_addr <= cfg_data;
                2'd2: length   <= cfg_data[15:0];
            endcase
        end
    end

    // DMA state machine
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state<=IDLE; bus_req<=0; dma_done<=0; cnt<=0;
        end else begin
            dma_done<=0; dma_ack<=0;
            case (state)
                IDLE: begin
                    if (dma_req) begin
                        dma_ack<=1; cnt<=0; bus_req<=1; state<=WAIT_GNT;
```

19.5 Simple 8-bit CPU with Interrupts

Verilog

```verilog
// Simplified 8-bit CPU — Fetch-Decode-Execute + Interrupt
module cpu8 (
    input         clk, rst_n,
    // Memory interface
    output reg [7:0] mem_addr,
    input      [7:0] mem_data,
    output reg [7:0] mem_wdata,
    output reg       mem_we,
    // Interrupt
    input            irq,
    output reg       irq_ack
);
    // Registers
    reg [7:0] acc;          // accumulator
    reg [7:0] pc;           // program counter
    reg [7:0] ir;           // instruction register
    reg [7:0] sp;           // stack pointer
    reg       ie;           // interrupt enable flag
    reg       carry_flag;

    // Opcodes
    localparam NOP  = 8'h00,  LOAD = 8'h01,  STORE= 8'h02;
    localparam ADD  = 8'h03,  AND  = 8'h04,  OR   = 8'h05;
    localparam JMP  = 8'h06,  JZ   = 8'h07,  EI   = 8'h08;
    localparam DI   = 8'h09,  RETI = 8'h0A;

    // States
    localparam [2:0] S_FETCH=0, S_DECODE=1, S_EXEC=2,
                     S_MEM=3, S_INT=4;
    reg [2:0] state;
    reg [7:0] operand;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            pc<=8'h00; acc<=0; sp<=8'hFF; ie<=0;
            irq_ack<=0; state<=S_FETCH; mem_we<=0;
        end else begin
            mem_we<=0; irq_ack<=0;
            case (state)
                S_FETCH: begin
                    mem_addr<=pc; ir<=mem_data; pc<=pc+1;
                    // Check for interrupt before decode
                    if (irq && ie) state<=S_INT;
                    else           state<=S_DECODE;
                end
```

19.6 I2C Master Controller (simplified)

Verilog

```verilog
module i2c_master (
    input        clk, rst_n,
    input        start_xfer,
    input  [6:0] slave_addr,
    input        rw,           // 0=write 1=read
    input  [7:0] wdata,
    output reg [7:0] rdata,
    output reg   done, ack_err,
    inout        sda,          // open-drain bidirectional
    output reg   scl
);
    // SCL divider: assume clk/4 = SCL frequency
    localparam [2:0] IDLE=0,START=1,ADDR=2,ACK=3,DATA=4,STOP=5;
    reg [2:0] state;
    reg [3:0] bit_cnt;
    reg [7:0] shift_reg;
    reg       sda_out, sda_oe;

    assign sda = sda_oe ? sda_out : 1'bz;  // open-drain

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            scl<=1; sda_out<=1; sda_oe<=1; state<=IDLE; done<=0;
        end else begin
            done<=0; ack_err<=0;
            case (state)
                IDLE: begin
                    scl<=1; sda_out<=1; sda_oe<=1;
                    if (start_xfer) begin
                        sda_out<=0; state<=START;  // START: SDA low while SCL high
                        shift_reg<={slave_addr, rw}; bit_cnt<=7;
                    end
                end
                START: begin
                    scl<=0; state<=ADDR;
                end
                ADDR: begin
                    sda_out<=shift_reg[7]; scl<=~scl;
                    if (scl) begin
                        shift_reg<={shift_reg[6:0],1'b0};
                        if (bit_cnt==0) begin sda_oe<=0; state<=ACK; end
                        else bit_cnt<=bit_cnt-1;
                    end
                end
                ACK: begin
```

19.7 VGA Sync Controller (640×480 @ 60 Hz)

Verilog

```verilog
module vga_sync (
    input        clk_25MHz, rst_n,     // 25.175 MHz pixel clock
    output reg   hsync, vsync,
    output reg   display_on,
    output reg [9:0] pixel_x, pixel_y
);
    // 640x480 @60Hz timing
    localparam H_ACTIVE=640, H_FP=16, H_SYNC=96,  H_BP=48;  // total=800
    localparam V_ACTIVE=480, V_FP=11, V_SYNC=2,   V_BP=31;  // total=524

    reg [9:0] h_cnt, v_cnt;

    always @(posedge clk_25MHz or negedge rst_n) begin
        if (!rst_n) begin
            h_cnt<=0; v_cnt<=0; hsync<=1; vsync<=1;
        end else begin
            // Horizontal counter
            if (h_cnt == H_ACTIVE+H_FP+H_SYNC+H_BP-1) begin
                h_cnt<=0;
                if (v_cnt==V_ACTIVE+V_FP+V_SYNC+V_BP-1) v_cnt<=0;
                else v_cnt<=v_cnt+1;
            end else h_cnt<=h_cnt+1;

            // Sync pulses (active low)
            hsync <= ~(h_cnt >= H_ACTIVE+H_FP && h_cnt < H_ACTIVE+H_FP+H_SYNC);
            vsync <= ~(v_cnt >= V_ACTIVE+V_FP && v_cnt < V_ACTIVE+V_FP+V_SYNC);

            display_on <= (h_cnt < H_ACTIVE) && (v_cnt < V_ACTIVE);
            pixel_x    <= h_cnt;
            pixel_y    <= v_cnt;
        end
    end
endmodule
```

Project Roadmap

Week	Project	Skills Mastered
1	Logic gates, Half/Full Adder	assign, module, ports
2	MUX 2:1, 4:1, Priority Encoder	case, ternary, casez
3	4-bit Ripple Carry Adder	Hierarchy, instantiation
4	D/JK/SR Flip-flops	posedge, negedge, async reset
5	N-bit Counter (up/down/mod)	Parameters, enable, modulo
6	8-bit Shift Register (SIPO/PISO)	Concatenation, serial I/O
7	8-bit ALU with flags	Arithmetic, overflow, carry
8	Traffic Light / Vending FSM	3-block FSM, state encoding
9	UART TX + RX	Baud rate, start/stop bits
10	SPI Master	CPOL/CPHA, shift register
11	I2C Master	Open-drain, ACK, bit-banging
12	Synchronous FIFO	Circular buffer, full/empty flags
13	Interrupt Controller	Priority encoder, masking, pending
14	DMA Controller	Bus arbitration, burst transfer
15-18	Simple 8-bit CPU	Fetch-Decode-Execute, ISR
19-22	VGA Controller + framebuffer	Pixel clock, sync timing
23-30	RISC-V RV32I Core (optional)	Pipeline, hazards, branch

20 Quick Reference Cheat Sheet

RTL Module Template

Verilog

```verilog
`timescale 1ns/1ps
`default_nettype none

module my_rtl #(
    parameter DW = 8,
    parameter AW = 4
) (
    input              clk,
    input              rst_n,
    input  [DW-1:0]    data_in,
    input              valid_in,
    output reg [DW-1:0] data_out,
    output reg          valid_out
);
    wire [DW-1:0] processed;
    assign processed = data_in + 1;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            data_out  <= {DW{1'b0}};
            valid_out <= 1'b0;
        end else begin
            data_out  <= processed;
            valid_out <= valid_in;
        end
    end
endmodule
```

Testbench Template

Verilog

```verilog
`timescale 1ns/1ps
module tb_my_rtl;
    reg  clk=0, rst_n=0, valid_in=0;
    reg  [7:0] data_in_t=0;
    wire [7:0] data_out_t;
    wire       valid_out_t;

    my_rtl #(.DW(8)) dut (
        .clk(clk), .rst_n(rst_n),
        .data_in(data_in_t), .valid_in(valid_in),
        .data_out(data_out_t), .valid_out(valid_out_t)
    );

    always #5 clk = ~clk;  // 100 MHz

    initial begin
        $dumpfile("out.vcd"); $dumpvars(0, tb_my_rtl);
        #22 rst_n=1;
        @(posedge clk); data_in_t=8'hAA; valid_in=1;
        @(posedge clk); data_in_t=8'h55;
        #50; $finish;
    end
endmodule
```

12 Golden Rules

#	Rule	Why It Matters
1	Use = in combinational always	Executes in order, no race
2	Use <= in clocked always	Simultaneous assignment, no race
3	Never mix = and <= in same block	Undefined simulation results
4	Use @(*) sensitivity list	Catches all inputs automatically
5	Always include default in case	Prevents unwanted latches
6	Assign defaults before if-else	No latch inference
7	`default_nettype none at top	Catches undeclared wires
8	Use named port mapping .port(sig)	Prevents silent mis-connections
9	output reg if driven by always	Language requirement
10	Parameters for all constants	Enables reuse
11	Write testbench before trusting design	Bugs caught in sim, not silicon
12	localparam for FSM state values	Prevents accidental override

---

Free Tools: EDA Playground (browser, no install) · Icarus Verilog + GTKWave (open-source) · Vivado WebPack (Xilinx FPGA) · Quartus Prime Lite (Intel FPGA)

■ **Tip: Simulate every concept the same day. 30 minutes of hands-on beats 3 hours of reading. If it does not simulate correctly, it will NOT work in hardware.**
